

Definition of RPC (Remote Procedure Call)

1. تعریف RPC

مفهوم RPC در واقع به پروتکل ارتباطی تو سیستم‌های توزیع شده است. کارش اینه که به برنامه ما اجازه میده به تابع رو روی یه ماشین یا سرور دیگه تو شبکه، دقیقاً همون طوری صدا بزنه و اجرا کنه که انگار داره به تابع لوکال رو روی سیستم خودش فراخوانی می‌کنه. هدف اصلی RPC اینه که پیچیدگی‌های شبکه رو از دید ما مخفی کنه. اینجوری دیگه نیازی نیست درگیر جزئیات پایین رده مثل ارسال و دریافت پکت‌ها یا مدیریت سوکت‌ها بشیم و می‌تونیم کدهای یه سیستم توزیع شده رو به همون راحتی برنامه‌های معمولی و یکپارچه بنویسیم.

2. ایده اصلی فراخوانی تابع از راه دور

ایده اصلی پشت RPC، ایجاد شفافیت تو سیستم‌های توزیع شده است. یعنی می‌خواد کاری کنه که فراخوانی به تابع تحت شبکه، هیچ فرقی با فراخوانی همون تابع روی حافظه سیستم خودمون نداشته باشه. با این ایده، توابع شبکه دقیقاً پشت رابط‌های برنامه‌نویسی آشنا مخفی میشن و برنامه‌نویس دیگه درگیر پیچیدگی‌های ارتباطی نمیشه؛ بلکه تمام تمرکزش رو میذاره روی توسعه منطق اصلی پروژه.

3. تفاوت فراخوانی محلی و فراخوانی راه دور

تو فراخوانی لوکال، تابع فراخوانی‌شونده و برنامه اصلی هر دو تو یک فضای حافظه مشترک هستن. مقادیر خیلی سریع از طریق استک پاس داده میشن و اگه خطایی پیش بیاد معمولاً کل برنامه با هم کرش می‌کنه. اما تو فراخوانی ریموت، تابع روی یک سرور دیگه با فضای حافظه کاملاً مجزا اجرا میشه. تو این حالت ما نمی‌تونیم رفرنس یا آدرس حافظه رو مستقیم پاس بدیم و تمام دیتا باید از بستر شبکه رد بشه. این یعنی سرعت اجرا به خاطر تاخیر شبکه

پایین‌تره و خطاهای جدیدی مثل قطعی ارتباط یا داون شدن سرور مقصد هم به سیستم اضافه میشه که تو حالت لوکال اصلا وجود ندارن.

4. نقش کلاینت استاب و سرور استاب

استاب‌ها مثل واسطه یا پروکسی عمل می‌کنن تا اون شفافیتی که تو ایده اصلی گفتیم رو برای برنامه‌نویس بسازن. کلاینت استاب سمت کلاینت قرار داره و خودش رو جای تابع اصلی جا می‌زنه. وقتی ما این تابع رو صدا می‌زنیم، کلاینت استاب پارامترها رو می‌گیره، اون‌ها رو به یک فرمت مناسب برای شبکه تبدیل و بسته‌بندی می‌کنه و می‌فرسته سمت سرور. سرور استاب تو سمت سرور منتظره. پیام رو از شبکه می‌گیره، بازش می‌کنه، تابع واقعی سرور رو با اون پارامترها اجرا می‌کنه و در نهایت خروجی رو دوباره بسته‌بندی می‌کنه و برمی‌گردونه به کلاینت استاب تا به دست برنامه ما برسه.

5. مفهوم Serialization و Deserialization

سریالایزیشن فرایندی است که در آن ساختارهای داده یا آبجکت‌های موجود در حافظه برنامه به یک فرمت خطی و استاندارد مثل جریانی از بایت‌ها تبدیل می‌شوند تا قابلیت ارسال روی بستر شبکه را پیدا کنند. دی‌سریالایزیشن دقیقا عملیات معکوس این فرایند است. یعنی جریانی از بایت‌ها که از شبکه دریافت شده است، دوباره بازسازی شده و به ساختار داده اولیه در حافظه سیستم مقصد تبدیل می‌شود. این دو عملیات تضمین می‌کنند که سیستم‌های مختلف با معماری‌های متفاوت بتوانند دیتای یکدیگر را به درستی درک کنند.

6. مفهوم IDL یا Interface Definition Language

زبان تعریف اینترفیس یک زبان میانی و کاملا مستقل از زبان‌های برنامه‌نویسی است که به عنوان قرارداد بین کلاینت و سرور عمل می‌کند. از آنجایی که در سیستم‌های توزیع‌شده ممکن است کلاینت و سرور با زبان‌های متفاوتی توسعه داده شوند، با استفاده از زبان تعریف اینترفیس ساختار توابع، نوع پارامترهای ورودی و نوع داده‌های خروجی به صورت

یکپارچه تعریف می‌شود. سپس ابزارهای توسعه با خواندن این قرارداد، کدهای واسط کلاینت استاب و سرور استاب را به صورت خودکار برای هر زبان تولید می‌کنند.

7. خطاهای رایج در RPC

در فراخوانی توابع تحت شبکه، سیستم با خطاهایی مواجه می‌شود که در حالت لوکال بی‌معنی هستند. از جمله این خطاها می‌توان به قطعی ارتباط شبکه، تایم اوت شدن کلاینت در انتظار پاسخ و از دسترس خارج شدن سرور مقصد اشاره کرد. پیچیده‌ترین خطا در این معماری، خرابی‌های جزئی است؛ به این معنا که درخواست اجرا می‌شود اما پاسخ در مسیر برگشت گم می‌شود. این مسئله باعث ایجاد چالش تکرارپذیری اجرای توابع می‌شود و سیستم باید مکانیزمی برای تشخیص درخواست‌های تکراری داشته باشد تا یک عملیات حساس بیش از یک بار اجرا نشود.

8. مقایسه RPC با REST

تفاوت اصلی این دو معماری در رویکرد آن‌ها به منابع و عملیات‌ها است. معماری رست منبع‌محور است و روی موجودیت‌ها تمرکز دارد و به طوری که از افعال استاندارد پروتکل اچ تی تی پی برای انجام عملیات روی آدرس‌ها استفاده می‌کند. در مقابل، معماری آر پی سی عمل‌محور است و مستقیماً روی فراخوانی توابع و متدها تمرکز دارد. معماری رست معمولاً از فرمت‌های متنی مانند جیسون استفاده می‌کند و برای رابط‌های برنامه‌نویسی عمومی یا همان پابلیک ای پی آی مناسب‌تر است. اما آر پی سی با استفاده از فرمت‌های باینری و سبک‌تر، کارایی بالاتری ارائه می‌دهد و بهترین انتخاب برای ارتباطات سریع و فشرده بین میکروسرویس‌های داخلی است.

9. مقایسه چند فناوری رایج RPC

فناوری‌های مختلفی برای پیاده‌سازی این معماری وجود دارند که هر کدام برای نیازمندی‌های خاصی طراحی شده‌اند. فناوری جی آر پی سی که توسعه‌یافته توسط گوگل است، از فرمت

باینری و زبان تعریف اینترفیس استفاده می‌کند و کارایی بسیار بالایی دارد. در سمت دیگر فناوری‌های قدیمی‌تر و ساده‌تری مانند جیسون آر پی سی و ایکس ام ال آر پی سی قرار دارند که مبتنی بر متن هستند، پیاده‌سازی آن‌ها ساده‌تر است اما امکانات پیشرفته‌ای مانند استریمینگ را پشتیبانی نمی‌کنند

بررسی چند فناوری

فناوری gRPC:

این فریم ورک اوپن سورس که گوگل توسعه اش داده الان یکی از مدرن ترین و پرکاربردترین تکنولوژی های آر پی سی تو سیستم های دیستریبیوتد به حساب میاد. این تکنولوژی به جای فرمت های متنی سنگین از پروتکل بافرز به عنوان آی دی ال و فرمت سریالایزیشن باینری استفاده می‌کند. این ویژگی در کنار استفاده از پروتکل اچ تی تی پی تو در لایه انتقال باعث میشه پیام ها به شدت فشرده و سریع منتقل بشن و امکاناتی مثل بای دایرکشنال استریمینگ خیلی راحت فراهم بشه. به همین خاطر جی آر پی سی انتخاب اول برای ارتباطات بین میکروسرویس هاست که پرفورمنس بالا نیاز دارن.

فناوری JSON-RPC:

این تکنولوژی به پروتکل خیلی سبک و استیت لس هست که برای تبادل دیتا از فرمت استاندارد متنی جیسون استفاده می‌کند. برخلاف جی آر پی سی اینجا نیازی به تعریف دقیق و سخت گیرانه اینترفیس ها یا همون آی دی ال نیست. این موضوع باعث میشه پیاده سازی و دیباگ کدها خیلی راحت تر بشه اما خب از اون طرف مزیت تایپ سیفتی رو موقع کامپایل از دست میدیم. چون این تکنولوژی خوانایی بالایی برای انسان داره و تو اکثر زبان ها و مرورگرها به صورت نیتیو ساپورت میشه معمولا برای ای پی آی های ساده تر و ارتباطات کلاینت سروری که سر بار پردازشی توشون دغدغه اصلی نیست کاربرد داره.

فناوری XML-RPC:

این پروتکل یکی از قدیمی ترین پیاده سازی های فراخوانی ریموت هست که از زبان مارک آپ متنی ایکس ام ال برای پکیج کردن دیتا و از پروتکل اچ تی تی پی برای انتقالشون استفاده میکنه. هرچند این پروتکل تو زمان خودش انقلاب بزرگی تو یکپارچه سازی سیستم ها کرد و پایه گذار پروتکل های پیچیده تری مثل سوپ شد اما الان به خاطر حجم بالای تگ ها و سربار زیاد تو سریالایزیشن و دی سریالایزیشن نسبت به جیسون و پروتوباف بیشتر به عنوان یه تکنولوژی لگسی شناخته میشه و عمدتا تو نگهداری سیستم های سازمانی قدیمی کاربرد داره.

محورهای مقایسه

حالا بر اساس 8 تا شاخصی که توی صورت پروژه گفته شده این فناوری هارو با هم مقایسه میکنیم:

نوع داده و فرمت پیام:

تکنولوژی gRPC از فرمت باینری استفاده میکنه که خیلی فشرده و سریعه. تکنولوژی JSON-RPC دیتا رو به صورت متن JSON میفرسته که خوندنش برای ما راحتتر ولی حجمش بیشتره. تکنولوژی XML-RPC هم از تگ های متنی XML استفاده میکنه که از همه سنگین تره و سربار بیشتری داره.

روش تعریف اینترفیس:

تو gRPC ما حتما باید اینترفیس ها رو با یه زبان IDL مشخص تعریف کنیم تا کلاینت و سرور همو بفهمن. اما JSON-RPC و XML-RPC اصلا نیازی به این کار ندارن و بدون IDL کار میکنن که باعث میشه ستاپ اولیه شون راحت تر باشه.

پشتیبانی از چند زبان برنامه‌نویسی:

هر سه تاشون از زبان‌های مختلف پشتیبانی می‌کنن اما gRPC به خاطر داشتن کامپایلر Protocol Buffers کدهای کلاینت و سرور رو برای اکثر زبان‌ها به صورت اتوماتیک جنریت می‌کنه. اون دوتای دیگه چون بیسشون متنی و استاندارد وب هست تو هر زبونی که بتونه JSON یا XML رو پارس کنه خیلی راحت قابل استفاده هستن.

کارایی و سربار ارتباطی:

از نظر پرفورمنس gRPC با اختلاف اوله چون دیتای باینری می‌فرسته و از پروتکل 2/HTTP استفاده می‌کنه که سربار شبکه رو به شدت کم می‌کنه. تکنولوژی JSON-RPC تو رتبه بعدیه و XML-RPC به خاطر ساختار تگ‌های طولانی‌ش بیشترین سربار و کمترین کارایی رو داره.

سادگی پیاده‌سازی:

تکنولوژی JSON-RPC از همه ساده‌تره چون فقط کافیه یه JSON بفرستیم و بگیریم. XML-RPC هم ساده است ولی کار با پارسرهای XML کمی دردسر داره. اما gRPC پیاده‌سازی سخت‌تری داره چون باید اول فایل‌های قرارداد رو بنویسیم و بعد کدها رو جنریت کنیم.

مناسب بودن برای سیستم‌های توزیع‌شده:

برای میکروسرویس‌های توزیع‌شده که بک‌اند هستن gRPC بهترین گزینه است چون تایپ‌سیفتی داره و خیلی سریعه. JSON-RPC برای ارتباطات سبک‌تر و کلاینت‌هایی که وب‌بیس هستن تو سیستم‌های توزیع‌شده مناسبه. XML-RPC هم دیگه الان برای سیستم‌های مدرن توزیع‌شده پیشنهاد نمیشه.

پشتیبانی از استریمینگ:

اینجا فقط gRPC هست که از استریمینگ پشتیبانی می‌کند. JSON-RPC و XML-RPC قابلیت استریمینگ ندارند و فقط برای ریکوئست و ریسپانس‌های ساده طراحی شدن.

موارد استفاده رایج:

تکنولوژی gRPC بیشتر برای ارتباطات داخلی بین میکروسرویس‌ها و سیستم‌هایی که پرفورمنس بالا می‌خوان استفاده میشه. JSON-RPC برای API های ساده و پابلیک یا ارتباط با کلاینت‌های وب کاربرد داره. XML-RPC هم معمولاً فقط وقتی استفاده میشه که مجبور باشیم با یه سیستم لگسی و قدیمی ارتباط بگیریم.

فناوری	فرمت داده	نیاز به IDL	پشتیبانی از استریمینگ	استقلال زبانی	سر بار ارتباطی	سادگی پیاده‌سازی	کاربرد رایج
gRPC	Protocol Buffers	بله	بله	بله (تولید کد خودکار)	بسیار کم	نسبتاً پیچیده	میکروسرویس‌های داخلی با کارایی بالا
JSON-RPC	JSON	خیر	خیر	بله (پشتیبانی گسترده JSON)	متوسط	بسیار ساده	API های ساده و ارتباطات سبک
XML-RPC	XML	خیر	خیر	بله	بسیار زیاد	متوسط	ارتباط با سیستم‌های لگسی و قدیمی

1. چرا RPC باعث می‌شود فراخوانی یک سرویس راه دور شبیه فراخوانی یک تابع محلی به نظر برسد؟

دلیل اصلی این شباهت استفاده از استاب‌ها است. کلاینت استاب در پس‌زمینه تمام پیچیدگی‌های مربوط به سریالایزیشن داده‌ها و ارتباطات شبکه را مخفی می‌کند و یک رابط برنامه‌نویسی دقیقاً مشابه با تابع اصلی در اختیار توسعه‌دهنده قرار می‌دهد.

2. چرا این شباهت می‌تواند خطرناک یا گمراه‌کننده باشد؟

این شفافیت باعث می‌شود توسعه‌دهنده فراموش کند که با یک ارتباط شبکه‌ای طرف است و از هزینه‌های زمانی و سربار شبکه غافل شود. به عنوان مثال اگر فراخوانی یک تابع آر پی سی درون یک حلقه تکرار قرار بگیرد، به دلیل تأخیر شبکه می‌تواند منجر به افت شدید پرفورمنس یا حتی از کار افتادن سیستم شود.

3. چه خطاهایی در فراخوانی راه دور ممکن است رخ دهد که در فراخوانی محلی وجود ندارد؟

مشکلاتی نظیر قطعی ارتباط شبکه، از دسترس خارج شدن سرور مقصد، تایم اوت شدن درخواست‌ها و از همه مهم‌تر خطاهای جزئی (مثل زمانی که درخواست اجرا می‌شود اما پاسخ در مسیر برگشت به دلیل مشکل شبکه گم می‌شود) که نیاز به مکانیزم‌های مدیریت خطای پیچیده‌تری دارند

4. در چه شرایطی RPC انتخاب بهتری از REST است؟

زمانی که ارتباطات درون شبکه‌ای بین میکروسرویس‌ها داریم و نیازمند سرعت بالا، تأخیر بسیار کم و فرمت انتقال داده فشرده مثل دیتای باینری هستیم. همچنین وقتی معماری سیستم بر پایه اجرای عملیات‌ها و اکشن‌های خاص طراحی شده باشد، آر پی سی بهترین انتخاب است.

5. در چه شرایطی REST انتخاب ساده‌تر یا مناسب‌تر است؟

زمانی که قصد توسعه یک ای پی آی عمومی برای استفاده مرورگرها یا کلاینت‌های خارجی را داریم. معماری رست به دلیل استفاده از استاندارد جهانی اچ تی تی پی، فرمت متنی خوانا مثل جیسون و تمرکز روی مدیریت و واکنشی منابع، پیاده‌سازی و درک بسیار ساده‌تری برای کلاینت‌ها دارد.